DEVOPS

Beginner Course Material

One Month | 4 Weeks | 20 Study Days

Learn by building: Linux, Git, CI/CD, Docker, cloud, IaC, Kubernetes, monitoring and DevSecOps.

Recommended pace: 2 hours per day - 45 minutes theory, 60 minutes lab, 15 minutes review.

Prepared as a practical beginner workbook. Commands may differ slightly across Windows, macOS, and Linux. Use a non-production practice environment.

How to Use This Course

This material assumes no professional DevOps experience. Complete the days in order because later labs reuse the same sample application and repository.

Learning method

- Read the concept section and rewrite key ideas in your own words.
- Run every command in a disposable practice environment.
- Keep all lab files in one Git repository.
- Record errors and solutions in a troubleshooting journal.
- Complete the daily assignment before moving forward.

Safety rules

- Never use production credentials or systems for practice.
- Read destructive commands before running them.
- Never commit passwords, tokens, private keys, or real .env files.
- Use least privilege and stop when a command target is unclear.

Day	Week	Topic
1	Week 1	DevOps Foundations and Setup
2	Week 1	Linux Essentials
3	Week 1	Shell Scripting Basics
4	Week 1	Networking for DevOps
5	Week 1	Git and GitHub Workflow
6	Week 2	Build and Dependency Basics
7	Week 2	CI/CD Concepts
8	Week 2	GitHub Actions Lab
9	Week 2	Testing and Quality Gates
10	Week 2	Configuration and Secrets
11	Week 3	Docker Fundamentals
12	Week 3	Docker Storage and Networking
13	Week 3	Docker Compose
14	Week 3	Cloud Fundamentals
15	Week 3	Infrastructure as Code Introduction
16	Week 4	Kubernetes Fundamentals
17	Week 4	Kubernetes Deployment Lab
18	Week 4	Monitoring and Logging
19	Week 4	DevSecOps and Reliability
20	Week 4	Capstone: Build and Deliver a Small Application

Week 1 - Foundations, Linux, Networking and Git

Weekly outcome: complete five connected lessons and commit all lab work to your learning repository.

Day 1: DevOps Foundations and Setup

Suggested time: 120 minutes **Level:** Beginner

Learning objectives

- Explain DevOps in plain language
- Understand the software delivery lifecycle
- Prepare a safe practice environment

Core concepts

DevOps is a culture and working method that helps development and operations teams deliver software frequently and reliably. It combines collaboration, automation, measurement, and continuous improvement.

A typical flow is Plan -> Code -> Build -> Test -> Release -> Deploy -> Operate -> Monitor. Feedback from production returns to planning. DevOps reduces handoffs; it does not remove responsibility.

Core ideas: version everything, automate repeatable work, make small changes, test early, observe systems, and learn from failures without blame.

Hands-on lab

Step 1. Install Git, VS Code, Docker Desktop (or Docker Engine), and create a GitHub account.

Step 2. Create a folder named devops-lab and a README.md describing your learning goal.

Step 3. Run version checks: git --version, docker --version, and code --version.

Daily assignment

Draw the DevOps lifecycle and write one example activity for each stage.

Knowledge check

1. What is the main purpose of devops foundations and setup?
2. Which command, file, or concept from today would you use first in a real problem?
3. What common mistake should a beginner avoid?
4. How would you verify that your work succeeded?

Notes

Day 2: Linux Essentials

Suggested time: 120 minutes **Level:** Beginner

Learning objectives

- Navigate Linux directories
- Manage files and permissions
- Inspect processes and system resources

Core concepts

Linux powers most servers and containers. The shell accepts commands; the filesystem starts at /. Important locations include /home, /etc, /var/log, /tmp, and /usr.

Permissions are represented as read (r), write (w), and execute (x) for owner, group, and others. Use least privilege and avoid running daily work as root.

Hands-on lab

Step 1. Practice pwd, ls -la, cd, mkdir, touch, cp, mv, rm, cat, less, head, tail, grep, and find.

Step 2. Create scripts/health.sh, make it executable with chmod +x, and print disk and memory usage.

Step 3. Inspect processes with ps aux and top.

```
pwd
ls -la
mkdir -p devops-lab/scripts
cd devops-lab
printf "hello devops\n" > notes.txt
cat notes.txt
df -h
free -h
```

Daily assignment

Create a Linux command cheat sheet with at least 20 commands and one example each.

Knowledge check

1. What is the main purpose of linux essentials?
2. Which command, file, or concept from today would you use first in a real problem?
3. What common mistake should a beginner avoid?
4. How would you verify that your work succeeded?

Notes

Day 3: Shell Scripting Basics

Suggested time: 120 minutes **Level:** Beginner

Learning objectives

- Use variables and conditions
- Automate repeatable commands
- Handle errors safely

Core concepts

A shell script is a text file containing commands. Begin Bash scripts with #!/usr/bin/env bash. Quote variables, validate inputs, and stop on unexpected failures.

Exit code 0 means success; non-zero means failure. Pipelines connect command output to another command. Environment variables configure programs without modifying code.

Hands-on lab

Step 1. Create backup.sh that accepts a source directory and creates a timestamped archive.

Step 2. Use set -euo pipefail, check that the input directory exists, and print a helpful error.

Step 3. Schedule a test command conceptually with cron; do not automate destructive actions.

```
#!/usr/bin/env bash
set -euo pipefail
echo "Host: $(hostname)"
echo "User: $(whoami)"
df -h /
```

Daily assignment

Write a script that reports hostname, current user, disk usage, memory usage, and current date.

Knowledge check

1. What is the main purpose of shell scripting basics?
2. Which command, file, or concept from today would you use first in a real problem?
3. What common mistake should a beginner avoid?
4. How would you verify that your work succeeded?

Notes

Day 4: Networking for DevOps

Suggested time: 120 minutes **Level:** Beginner

Learning objectives

- Understand IP, ports and DNS
- Test network connectivity
- Explain HTTP requests

Core concepts

An IP address identifies a network interface. A port identifies a service on a host. DNS converts names into IP addresses. Common ports include 22 SSH, 80 HTTP, and 443 HTTPS.

HTTP uses methods such as GET, POST, PUT, PATCH, and DELETE. Status codes: 2xx success, 3xx redirect, 4xx client issue, and 5xx server issue.

Hands-on lab

Step 1. Use ip addr (or ipconfig), ping, nslookup, curl -I, and ss -tulpn.

Step 2. Run a local web server and access it through localhost and its port.

Step 3. Use curl to inspect headers and a JSON response from a public test API if internet access is available.

Daily assignment

Explain why localhost:3000 and localhost:8080 can run different applications.

Knowledge check

1. What is the main purpose of networking for devops?
2. Which command, file, or concept from today would you use first in a real problem?
3. What common mistake should a beginner avoid?
4. How would you verify that your work succeeded?

Notes

Day 5: Git and GitHub Workflow

Suggested time: 120 minutes **Level:** Beginner

Learning objectives

- Track changes with Git
- Work with branches
- Resolve a simple merge conflict

Core concepts

Git stores snapshots of a project. The working tree contains edits, the staging area selects the next snapshot, and a commit records it. Remote repositories support collaboration and backup.

Use short-lived branches and meaningful commits. Pull before pushing, review changes with git diff, and never commit secrets.

Hands-on lab

Step 1. Initialize devops-lab, configure identity, add README.md, commit, and inspect git log.

Step 2. Create a feature branch, change the README, merge it into main, and delete the branch.

Step 3. Create and resolve a controlled merge conflict.

```
git init
git status
git add README.md
git commit -m "docs: add learning plan"
git switch -c feature/notes
git log --oneline --graph --all
```

Daily assignment

Publish the repository and include a clear README with prerequisites and commands.

Knowledge check

1. What is the main purpose of git and github workflow?
2. Which command, file, or concept from today would you use first in a real problem?
3. What common mistake should a beginner avoid?
4. How would you verify that your work succeeded?

Notes

Week 2 - Build, CI/CD, Testing and Secrets

Weekly outcome: complete five connected lessons and commit all lab work to your learning repository.

Day 6: Build and Dependency Basics

Suggested time: 120 minutes **Level:** Beginner

Learning objectives

- Understand builds and artifacts
- Use package managers safely
- Separate configuration from code

Core concepts

A build converts source into a runnable artifact. Examples include a Java JAR, compiled binary, or bundled web application. Reproducible builds use locked dependency versions.

Dependencies should be declared in files such as package.json, requirements.txt, or pom.xml. Artifacts should be versioned and promoted between environments rather than rebuilt differently.

Hands-on lab

Step 1. Create a minimal Node.js or Python web application.

Step 2. Install dependencies, run locally, and document the start command.

Step 3. Add a .gitignore and ensure generated files and secrets are excluded.

Daily assignment

Describe source code, dependency, build artifact, and runtime configuration for your app.

Knowledge check

1. What is the main purpose of build and dependency basics?
2. Which command, file, or concept from today would you use first in a real problem?
3. What common mistake should a beginner avoid?
4. How would you verify that your work succeeded?

Notes

Day 7: CI/CD Concepts

Suggested time: 120 minutes **Level:** Beginner

Learning objectives

- Differentiate CI, delivery and deployment
- Design a basic pipeline
- Recognize pipeline gates

Core concepts

Continuous Integration automatically validates small code changes. Continuous Delivery keeps a releasable artifact ready; Continuous Deployment releases every validated change automatically.

A beginner pipeline can run checkout -> install -> lint -> test -> build -> package. Deployment should use approvals where risk requires them and provide rollback.

Hands-on lab

Step 1. Write your application pipeline as a diagram.

Step 2. Define pass/fail criteria for lint, unit tests, build, and security checks.

Step 3. Create one intentional test failure and observe how it should stop the pipeline.

Daily assignment

Explain the difference between continuous delivery and continuous deployment.

Knowledge check

1. What is the main purpose of ci/cd concepts?
2. Which command, file, or concept from today would you use first in a real problem?
3. What common mistake should a beginner avoid?
4. How would you verify that your work succeeded?

Notes

Day 8: GitHub Actions Lab

Suggested time: 120 minutes **Level:** Beginner

Learning objectives

- Create a workflow file
- Run tests automatically
- Read workflow logs

Core concepts

GitHub Actions workflows are YAML files in .github/workflows. A workflow reacts to events, contains jobs, and each job contains steps. Runners execute the steps.

Pin trusted actions to stable versions, use repository secrets for credentials, and grant the minimum token permissions.

Hands-on lab

Step 1. Create ci.yml triggered by pushes and pull requests.

Step 2. Checkout code, set up the runtime, install dependencies, run tests, and build.

Step 3. Add a status badge to the README after the workflow succeeds.

```
name: CI
on: [push, pull_request]
jobs:
test:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- uses: actions/setup-node@v4
with:
node-version: 22
- run: npm ci
- run: npm test
- run: npm run build
```

Daily assignment

Submit a screenshot or link showing one successful and one failed workflow run.

Knowledge check

1. What is the main purpose of github actions lab?

2. Which command, file, or concept from today would you use first in a real problem?

3. What common mistake should a beginner avoid?

4. How would you verify that your work succeeded?

Notes

Day 9: Testing and Quality Gates

Suggested time: 120 minutes **Level:** Beginner

Learning objectives

- Understand testing levels
- Add a quality gate
- Make failures actionable

Core concepts

Unit tests validate small pieces, integration tests validate connected components, and end-to-end tests validate user flows. A pipeline should fail clearly when a required check fails.

Good logs state what failed, where it failed, and how to reproduce it. Flaky tests reduce trust and should be fixed rather than repeatedly retried.

Hands-on lab

Step 1. Add at least two unit tests to the sample application.

Step 2. Make the CI job publish a test summary.

Step 3. Introduce a failing test, inspect the log, then fix it.

Daily assignment

Write acceptance criteria for one application feature and map each criterion to a test.

Knowledge check

1. What is the main purpose of testing and quality gates?
2. Which command, file, or concept from today would you use first in a real problem?
3. What common mistake should a beginner avoid?
4. How would you verify that your work succeeded?

Notes

Day 10: Configuration and Secrets

Suggested time: 120 minutes **Level:** Beginner

Learning objectives

- Separate environments
- Protect sensitive values
- Use environment variables

Core concepts

Configuration differs by environment; source code should not. Store non-secret configuration as environment variables or config files. Store secrets in an approved secret manager or CI secret store.

Never commit passwords, tokens, private keys, or .env files containing real credentials. Rotate any secret exposed in Git history.

Hands-on lab

Step 1. Read an APP_PORT environment variable with a safe default.

Step 2. Create .env.example containing names but no real secrets.

Step 3. Add .env to .gitignore and configure a fake CI secret for practice.

Daily assignment

Create a checklist for preventing secret leakage before every commit.

Knowledge check

1. What is the main purpose of configuration and secrets?
2. Which command, file, or concept from today would you use first in a real problem?
3. What common mistake should a beginner avoid?
4. How would you verify that your work succeeded?

Notes

Week 3 - Containers, Cloud and Infrastructure as Code

Weekly outcome: complete five connected lessons and commit all lab work to your learning repository.

Day 11: Docker Fundamentals

Suggested time: 120 minutes **Level:** Beginner

Learning objectives

- Explain images and containers
- Write a Dockerfile
- Run and inspect a container

Core concepts

An image is an immutable package containing application files and runtime dependencies. A container is a running instance of an image. A registry stores and distributes images.

Dockerfile layers are cached. Copy dependency files and install dependencies before copying frequently changing source code. Use a small trusted base image and a non-root user where possible.

Hands-on lab

Step 1. Write a Dockerfile for the sample application.

Step 2. Build with docker build -t beginner-app:1.0 .

Step 3. Run with docker run --rm -p 8080:3000 beginner-app:1.0 and test with curl.

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
USER node
EXPOSE 3000
CMD ["npm", "start"]
```

Daily assignment

Document every Dockerfile instruction used and explain why it is needed.

Knowledge check

1. What is the main purpose of docker fundamentals?
2. Which command, file, or concept from today would you use first in a real problem?
3. What common mistake should a beginner avoid?
4. How would you verify that your work succeeded?

Notes

Day 12: Docker Storage and Networking

Suggested time: 120 minutes **Level:** Beginner

Learning objectives

- Use ports and networks
- Persist data with volumes
- Inspect container behavior

Core concepts

Port publishing maps a host port to a container port. User-defined networks let containers find one another by name. Volumes persist data beyond a container lifecycle.

Containers should be disposable. Logs should normally go to standard output, and state should live in an appropriate external service or volume.

Hands-on lab

Step 1. Inspect container logs and metadata.

Step 2. Create a named volume and verify data survives container recreation.

Step 3. Create a user-defined network and connect two practice containers.

Daily assignment

Explain the difference between an image, container, volume, and network.

Knowledge check

1. What is the main purpose of docker storage and networking?
2. Which command, file, or concept from today would you use first in a real problem?
3. What common mistake should a beginner avoid?
4. How would you verify that your work succeeded?

Notes

Day 13: Docker Compose

Suggested time: 120 minutes **Level:** Beginner

Learning objectives

- Define multi-container applications
- Manage services together
- Use health checks and dependencies

Core concepts

Compose describes services, networks, and volumes in YAML. It is useful for local development and small controlled environments.

Service names become DNS names on the Compose network. depends_on controls startup order but readiness usually requires a health check or retry logic.

Hands-on lab

Step 1. Create compose.yaml for the app and a database or simple supporting service.

Step 2. Use environment variables without committing secrets.

Step 3. Run docker compose up -d, inspect logs, and stop with docker compose down.

```
services:
app:
build: .
ports:
- "8080:3000"
environment:
APP_PORT: 3000
restart: unless-stopped
```

Daily assignment

Add a health check and document how to verify every service is ready.

Knowledge check

1. What is the main purpose of docker compose?
2. Which command, file, or concept from today would you use first in a real problem?
3. What common mistake should a beginner avoid?
4. How would you verify that your work succeeded?

Notes

Day 14: Cloud Fundamentals

Suggested time: 120 minutes **Level:** Beginner

Learning objectives

- Explain IaaS, PaaS and SaaS
- Understand regions and availability zones
- Estimate basic operational responsibilities

Core concepts

Cloud platforms provide on-demand compute, storage, networking, databases, and managed services. IaaS offers more control; PaaS reduces infrastructure management; SaaS delivers a complete application.

Design for failure: avoid a single point of failure, back up important data, monitor cost, and apply least privilege. The shared responsibility model divides security duties between provider and customer.

Hands-on lab

Step 1. Map the sample application to compute, network, storage, DNS, and monitoring services.

Step 2. Compare hosting it on a virtual machine versus a managed application platform.

Step 3. Write three cost-control rules for a learning account.

Daily assignment

Choose IaaS or PaaS for the sample app and justify the decision.

Knowledge check

1. What is the main purpose of cloud fundamentals?
2. Which command, file, or concept from today would you use first in a real problem?
3. What common mistake should a beginner avoid?
4. How would you verify that your work succeeded?

Notes

Day 15: Infrastructure as Code Introduction

Suggested time: 120 minutes **Level:** Beginner

Learning objectives

- Explain declarative infrastructure
- Understand Terraform workflow
- Manage state carefully

Core concepts

Infrastructure as Code stores desired infrastructure in version-controlled files. Declarative tools calculate changes needed to reach the desired state.

Terraform commonly uses init, fmt, validate, plan, and apply. State maps configuration to real resources and must be protected. Never store secrets in plain text state or public repositories.

Hands-on lab

Step 1. Install Terraform or review an instructor-provided environment.

Step 2. Create a local-file resource or another no-cost practice resource.

Step 3. Run fmt, validate, plan, apply, inspect the result, and destroy only the lab resource.

```
terraform init
terraform fmt
terraform validate
terraform plan
terraform apply
# When finished, remove only lab resources:
terraform destroy
```

Daily assignment

Explain why reviewing a plan is necessary before applying infrastructure changes.

Knowledge check

1. What is the main purpose of infrastructure as code introduction?
2. Which command, file, or concept from today would you use first in a real problem?
3. What common mistake should a beginner avoid?
4. How would you verify that your work succeeded?

Notes

Week 4 - Kubernetes, Observability, Security and Capstone

Weekly outcome: complete five connected lessons and commit all lab work to your learning repository.

Day 16: Kubernetes Fundamentals

Suggested time: 120 minutes **Level:** Beginner

Learning objectives

- Explain cluster components
- Understand Pods and Deployments
- Use kubectl safely

Core concepts

Kubernetes schedules container workloads on a cluster. The control plane manages desired state; worker nodes run Pods. A Deployment manages replicated Pods and rolling updates. A Service provides stable access.

Declarative manifests describe desired state. Labels select related objects. Readiness controls traffic; liveness detects when a container should restart.

Hands-on lab

Step 1. Use a local cluster such as kind, minikube, or Docker Desktop Kubernetes.

Step 2. Inspect nodes and namespaces with kubectl get.

Step 3. Create a Deployment for a small web image and inspect Pods.

```
kubectl get nodes
kubectl get pods -A
kubectl create deployment beginner-app --image=nginx:alpine
kubectl get deployments,pods
kubectl describe deployment beginner-app
```

Daily assignment

Draw the relationship between Cluster, Node, Pod, Deployment, and Service.

Knowledge check

1. What is the main purpose of kubernetes fundamentals?

2. Which command, file, or concept from today would you use first in a real problem?

3. What common mistake should a beginner avoid?

4. How would you verify that your work succeeded?

Notes

Day 17: Kubernetes Deployment Lab

Suggested time: 120 minutes **Level:** Beginner

Learning objectives

- Expose an application
- Scale and update a Deployment
- Diagnose basic failures

Core concepts

A Service selects Pods by labels and exposes a stable virtual address. Rolling updates gradually replace old Pods. Rollback returns to an earlier ReplicaSet when an update is unhealthy.

Start troubleshooting with kubectl get, describe, logs, and events. Check image names, environment variables, ports, probes, labels, and resource availability.

Hands-on lab

Step 1. Create deployment.yaml and service.yaml for the sample app.

Step 2. Scale to three replicas and confirm Pods are distributed as available.

Step 3. Change the image tag, observe rollout status, view history, and practice rollback.

```
kubectl scale deployment beginner-app --replicas=3
kubectl rollout status deployment/beginner-app
kubectl rollout history deployment/beginner-app
kubectl logs deployment/beginner-app
```

Daily assignment

Write a five-step Kubernetes troubleshooting checklist.

Knowledge check

1. What is the main purpose of kubernetes deployment lab?
2. Which command, file, or concept from today would you use first in a real problem?
3. What common mistake should a beginner avoid?
4. How would you verify that your work succeeded?

Notes

Day 18: Monitoring and Logging

Suggested time: 120 minutes **Level:** Beginner

Learning objectives

- Distinguish metrics, logs and traces
- Define useful alerts
- Create a simple health view

Core concepts

Observability helps explain internal system behavior from outputs. Metrics show numeric trends, logs record events, and traces follow requests across services.

Monitor latency, traffic, errors, and saturation. Alerts should be actionable and tied to user impact. Dashboards support investigation but do not replace alerts.

Hands-on lab

Step 1. Add a /health endpoint to the sample app.

Step 2. Produce structured logs with timestamp, level, and message.

Step 3. Define four metrics and two alerts for the application.

Daily assignment

Write an alert message that includes impact, evidence, and a first troubleshooting action.

Knowledge check

1. What is the main purpose of monitoring and logging?
2. Which command, file, or concept from today would you use first in a real problem?
3. What common mistake should a beginner avoid?
4. How would you verify that your work succeeded?

Notes

Day 19: DevSecOps and Reliability

Suggested time: 120 minutes **Level:** Beginner

Learning objectives

- Shift security left
- Apply least privilege
- Plan backup and rollback

Core concepts

DevSecOps integrates security into everyday delivery. Scan dependencies and images, protect branches, review infrastructure changes, and patch supported versions.

Reliability includes capacity, health checks, backups, rollback, incident response, and learning. A blameless review focuses on system improvements.

Hands-on lab

Step 1. Run an available dependency audit and review findings.

Step 2. Confirm the container runs without unnecessary privileges.

Step 3. Write a rollback plan and a simple incident timeline template.

Daily assignment

Create a release checklist covering security, backup, deployment, validation, and rollback.

Knowledge check

1. What is the main purpose of devsecops and reliability?
2. Which command, file, or concept from today would you use first in a real problem?
3. What common mistake should a beginner avoid?
4. How would you verify that your work succeeded?

Notes

Day 20: Capstone: Build and Deliver a Small Application

Suggested time: 120 minutes **Level:** Beginner

Learning objectives

- Combine the month’s skills
- Demonstrate repeatable delivery
- Present evidence and lessons learned

Core concepts

The capstone proves that you can move a small application from source control to an automated, containerized delivery workflow. Keep the solution small, documented, and reproducible.

Required evidence: repository, README, tests, passing CI workflow, Dockerfile, Compose or Kubernetes manifests, health check, sample logs, and rollback instructions.

Hands-on lab

Step 1. Create or finalize a small web application with / and /health endpoints.

Step 2. Push code through a branch and pull request. Run automated tests and build an image.

Step 3. Deploy locally with Compose or Kubernetes, verify health, simulate one failure, recover, and record results.

Daily assignment

Prepare a five-minute demonstration and a one-page reflection: what worked, what failed, and what you would improve.

Knowledge check

1. What is the main purpose of capstone: build and deliver a small application?
2. Which command, file, or concept from today would you use first in a real problem?
3. What common mistake should a beginner avoid?
4. How would you verify that your work succeeded?

Notes

Capstone Assessment Rubric

Use this rubric to evaluate the final project. A passing beginner submission should score at least 70/100 and must not expose secrets.

Area	Evidence	Points
Source control	Clear commits, branches, README and .gitignore	15
Application quality	Runs locally, health endpoint, tests	15
CI pipeline	Automated install, test and build	20
Container	Working Dockerfile and reproducible run command	15
Deployment	Compose or Kubernetes manifests and verification	15
Operations	Logs, health check, monitoring ideas and rollback	10
Security	No secrets; dependency/image review; least privilege	10

Final demonstration checklist

- Clone the repository on a clean machine or folder.
- Follow the README without undocumented steps.
- Run tests and show a passing pipeline.
- Build and run the container.
- Deploy, verify health, inspect logs, and explain rollback.
- Describe one failure encountered and the lesson learned.

Essential Command Reference

Linux

```
pwd, ls -la, cd, mkdir, cp, mv, rm, grep, find, chmod, ps, top, df -h, free -h, tail -f
```

Git

```
git status, git diff, git add, git commit, git switch, git merge, git pull, git push, git log
```

Docker

```
docker build, docker images, docker run, docker ps, docker logs, docker exec, docker inspect, docker stop, docker compose up/down
```

Kubernetes

```
kubectl get, describe, logs, apply, delete, scale, rollout status/history/undo
```

Terraform

```
terraform init, fmt, validate, plan, apply, output, state list, destroy
```

Glossary

Artifact	A versioned output produced by a build.
CI	Automatic integration and validation of code changes.
CD	Automated preparation or release of validated changes.
Container	A running isolated process created from an image.
Deployment	A Kubernetes object that manages replicated Pods and updates.
Idempotent	Safe to repeat while producing the same desired result.
Image	An immutable package used to start containers.
Pipeline	An automated sequence of delivery stages.
Rollback	Returning a system to a known earlier version.
Secret	Sensitive information such as a password, token, or key.
Service	A stable network endpoint for a set of Kubernetes Pods.
Version control	A system that records and coordinates file changes.

Next steps after this month

- Repeat the capstone using a different application stack.
- Learn one cloud provider through its official beginner path.
- Study deeper Linux administration, networking, and security.
- Add infrastructure code, centralized monitoring, and a managed secret store.
- Build a portfolio that explains decisions and troubleshooting, not only successful screenshots.